

Кейсы

[Критерии оценивания](#)

[Security gate для данных](#)

[Безопасная MLOps система](#)

[Безопасная архитектура памяти агентов](#)

[Цензор-модуль для изображений](#)

Критерии оценивания

Бизнес-ценность

Нужно показать актуальность кейса, какую бизнес-задачу он решает, почему она важна именно сейчас и для каких процессов компании это может быть полезно.

Дополнительным преимуществом будет демонстрация, почему именно представляемое решение нужно купить бизнесу.

Глубина проработки кейса

Команда должна показать, что погрузилась во все аспекты предлагаемого решения: предметную область, пользователей, ограничения, риски, зависимости, критические пути и технологии (один из важнейших критериев, чтобы ребята попредлагали ракетсаенс). Важно, чтобы были собраны данные из индустрии, изучены лучшие практики, проработаны сценарии использования.

Аналоги

Показать, насколько широко был изучен ландшафт качественных аналогов. Затронули существующие продукты, технологии, подходы, статьи, исследования и практики из индустрии. По каким критериям оценивались их решение, в чем их сильные и слабые стороны, что команда взяла в свой кейс и почему предложенный подход лучше для задачи кейса.

Результативность

Необходимо показать, что команда продумала метрики, на которые имеет влияние решение, и связала их с метриками зависимых процессов. Важно описать, как будет измеряться эффект, какие показатели должны измениться после внедрения, какие риски остаются и как будет приниматься решение об их допустимости. Польза решения должна быть очевидна.

Качество презентации

Необходимо показать, насколько ясно, структурированно и убедительно команда донесла свою идею. Оцениваются логика повествования, визуальное оформление материалов, качество аргументации, способность выделить ключевые выводы и удерживать фокус на бизнес-ценности решения. Важно, чтобы презентация была понятна как экспертам предметной области, так и аудитории без глубокого погружения в тему.

Security gate для данных

Куратор: Акылжан Керимбай (tg @akilph)

Контекст бизнеса

Компании обучают десятки ML-моделей на различных данных. Обучающие выборки часто формируются как промежуточные артефакты — parquet-файлы, выгрузки, локальные сэмплы, csv — и попадают в обучение без контроля происхождения, целостности, состава и безопасности. Необходимо спроектировать «Security Gate» для проверки безопасности наборов данных.

В качестве конкретного примера рассматривается ML-контур крупного банка как одна из самых зрелых и регулируемых сред, где описанные риски проявляются особенно остро. Архитектура типична для банковской ML-инфраструктуры.

Образ результата

Концепция:

- Разработан концепт единой точки контроля обучающих датасетов – Security Gate для ML-процессов банка.

Архитектура:

- Security Gate интегрируется в существующий ML-пайплайн как обязательный этап перед обучением модели.
- Gate работает в режиме pre-training data validation: подключается к источникам обучающих артефактов (файлы, таблицы, витрины) через API или внутри workflow оркестрации.
- Использует набор валидаторов и детекторов для автоматического анализа датасетов.

Функционал прототипа:

- Проверка датасета по критериям
- Семантические и статистические аномалии.
- Признаки отравления (poisoning): аномальные вкрапления, выбросы, некорректная разметка, шаблоны известных атак.
- Code execution payloads: поиск вредоносных конструкций/обфускаций в строковых признаках (например, code injection).
- Выполнение внутренней политики допустимости и состава данных.

Формирование отчёта:

- Генерирует машиночитаемый и человекочитаемый отчёт с результатами анализа, детектированными инцидентами, метриками стабильности, списком подозрительных признаков.
- На основе политики и результатов анализа Security Gate формирует заключение (allow/block) и возвращает его в пайплайн, запрещая или разрешая продвижение модели к этапу обучения/дальнейшей эксплуатации.

Безопасная MLOps система

Куратор: Никита Барсуков (tg @lainisourgod)

Контекст бизнеса

Бизнесы хотят ML. ML приносит деньги. Но ML приносит деньги ровно до того, как он ломается. И тогда он перестает приносить деньги. Деньги теряются. Мы хотим больше денег. И хотим их 24/7.

Здесь на сцену выходит MLOps. MLOps все автоматизирует, делает воспроизводимым, надежным, объяснимым, и удобным. Бизнес любит MLOps. Там где, DS скажет "у меня на ноуте в `model_two_pd_new_features_backup_prod_v2_v3.ipynb` метрика `f@k3kr=0.7843`", MLOps скажет "все работает, деньги текут, спи спокойно".

Но MLOps не всесилен. MLOps знает о безопасности только то, что не стоит хранить секреты в коде. Это правда, но этого явно мало. Ему нужен он. Ему нужна она. Ему нужны они. **MLSecOps**-ы. Они помогут не подорваться на пороховой бочке совершенно безопасной модельки `sharegpt_vicuna_unfiltered` от `anon8231489123`. Они защитят от бесов по имени `pytirch` и `tenserflew`. Они поймают злых конкурентов, которые отравляют наши модели. Там где MLOps делает твою систему надежной, MLOps делает ее неуязвимой. Мы любим MLOps. Вы любите MLOps. Вы станете MLOps, и спасете мир.

MLOps живет поверх MLOps. Мы видим пайплайн, и поэтому можем видеть угрозы в нем. Мы знаем, что у нас в проде, и так понимаем, что будут атаковать. Мы видим актуальное состояние системы, и можем ловить инциденты за наносекунды после их наступления. Мы не патчим дыры, а *управляем* системой.

Но что мы видим в мире? В MLflow миллионы уязвимостей и нет RBAC. Прод стоит на либах пятилетней давности с десятками уязвимостей. DS уволился 3 года назад, и никто не может перевести модель.

Образ результата

Собрать из блоков безопасную MLOps-систему и склеить практиками MLOps. Помогите Игнатию перестать терять деньги на инцидентах. Постройте свою модель угроз — что плохого может произойти в системе, — и докажите, что вы закрыли эту боль. Сделайте CI/CD с Security Gate-ами. Постройте реестр моделей с полной видимостью ваших крутых безопасных фишек. Покройте весь жизненный цикл ML-решений от проработки требований до вывода из эксплуатации. Покажите разные модели в обучении и эксплуатации. Сделайте удобно, чтобы безопасность была автоматической фишкой системы.

Модель Угроз

- Это паспорт безопасности системы. Должна содержать как минимум активы, угрозы, последствия и ваши митигации/практики.

- Если запаритесь — еще поверхность атаки, уязвимость, и риски (по лекции Саши "Модель Угроз").
- Должна покрывать весь жизненный цикл систем ИИ (возьмите в лекции Гамида "Введение в безопасность ИИ").
- Приоритизируйте и обоснуйте, на чем вы сосредоточились в первую очередь, а что оставили на потом. Даю вам свободу поресерчить и подумать (или засунуть лекции в своего ИИ-ассистента и спросить его саммари).
- Покажите свое мышление — мы будем читать МУ глазами перед демо, так что свой аи слоп хотя бы причешите.

Задеплоенные модельки

- Как минимум 3 разных ML-решения на ваш выбор. Возьмите с HF, кеггла или гитхаба.
- Это могут быть в том числе агентские системы, но больше фокус на не(де)генеративный ИИ.
- Доп балл — базовая runtime-защита от разных атак.

Полноценная система

- В первую очередь — работающий MVP с реестром моделей и профилем безопасности системы. Затем низковисящие фрукты. А после — полный practice maxxing.
- По возможности используйте уже готовые решения, анализируя их безопасность, и докручивая нужные фишки (знаете как в mlflow добавить gbas в одну команду? а может просто его не использовать?).
- Реализуйте любой UI, который успеете — хоть вебка, хоть отчет в CI, хоть cli утилитки, но полноценная видимость в вебе лучше всего.
- Покажите роли в системе — DS-ы, MLSecOps-ы, DE-ы, продакты, ..., отчет для CEO где захардкожено "✅ все супер".

Демо

- Покажите разные сценарии работы в вашей системе — затягивание внешней модельки, обучение, деплой, вывод из эксплуатации.
- В вашей системе должна быть история событий. Никто не забыт. Ничего не будут упущено.
- Все должно подниматься на ноуте через один docker compose, демо-сценарии тоже должны легко запускаться.
- Если добавите в код закладки с RCE — не прощу 🐼.

Дополнительные материалы

Сага о ИБАНКе <https://claude.ai/public/artifacts/d4beb643-cd06-4aa9-9bd5-8a434fb72ca9>.

В ней есть большой список проблем и практик, можете вдохновляться или идти по списку.

HiddenLayer <https://www.hiddenlayer.com/>. Пример для вдохновения. Хорошая контора с кучей репортов на уязвы в ML-пайплайнах. Сделали свою платформу.

Безопасная архитектура памяти агентов

Куратор: Тимур Низамов (tg @nizamovtimur)

Тестовый стенд – DeepResearch с памятью:

<https://github.com/nizamovtimur/deep-research-memory>

Контекст бизнеса

В современных ГенИИ-системах используется долговременная память пользователя:

- хранение предпочтений, контекста и истории взаимодействий
- персонализация ответов и действий агента
- накопление знаний о пользователе для повышения качества сервиса
- Память реализуется как внешнее хранилище (vector DB / graph / hybrid memory), которое агент читает и обновляет автоматически.

Долговременная память AI-агентов создаёт новый класс угроз:

- Memory poisoning
 - внедрить ложные факты или prompt-injection для DoS
 - подменить пользовательские предпочтения
 - исказить поведение агента
- Cross-tenant leakage
 - данные одного пользователя могут попасть в контекст другого
 - embeddings могут “смешивать” семантически похожие, но приватные данные
 - отсутствие строгой изоляции превращает memory в канал утечки
- Sensitive data retention
 - персональные данные (ПДн)
 - секреты (API keys, credentials)
 - конфиденциальные бизнес-данные

Образ результата

1. **Формализованная модель угроз**
 - Типы злоумышленников: пользователь, внешние данные, внедрение в модель.
 - Поверхности атаки: ввод данных, хранение, извлечение.
 - Активы: записи памяти, эмбединги, профили пользователей.
2. **Набор тестов**
 - Инъекция запросов в память.
 - Семантическая отравление.
 - Утечка данных между пользователями (cross-tenant leakage).
 - Манипуляция предпочтениями/действиями в ответах.
3. **Демонстрация атак**
 - Внедрение через сохранённую запись памяти.
 - Симуляция утечки данных между пользователями.
4. **Защитные механизмы**
 - Ограничения доступа к памяти.
 - Фильтрация и валидация входящих запросов.

- Защита памяти от модификаций (обеспечение целостности).
- Мониторинг и откат подозрительных изменений.
- Особое внимание к архитектуре, чтобы безопасность была базой, а не "заплаткой".

5. Оценка эффективности

- Доля успешных атак до и после защиты.
- Точность и полнота извлечения памяти (precision/recall).

Результаты будут использованы для:

- проектирования безопасного memory layer для AI-агентов
- внедрения trust-aware retrieval архитектуры
- разработки guardrails production AI систем
- улучшения изоляции пользовательских данных в multi-tenant системах

Дополнительные материалы

Исследования по теме:

- <https://genai.owasp.org/2026/05/13/memory-is-a-feature-it-is-also-an-attack-surface/>
- <https://arxiv.org/pdf/2509.26354>
- <https://github.com/ShaoShuai0605/Misevolution>
- <https://github.com/m-pentest/memory-poisoning-demo>

Цензор-модуль для изображений

Куратор: Никита Иванов (tg [@ikitat](#))

Контекст бизнеса

Банк внедряет ИИ-систему генерации изображений и закрывает ею три сценария: text2image (картинка по текстовому запросу), img2img рестайлинг (смена стиля) и img2img редактирование (точечные правки).

Генеративные модели непредсказуемы: даже безобидный запрос может дать недопустимый результат, а пользователь «в порыве креатива» – намеренно или случайно – вытащить из модели контент, за который банк оштрафуют, получит репутационный удар или лишится лицензии.

Запрос бизнеса: независимое средство защиты информации – цензор-модуль (Гардрейл для картинок), который не пропускает запрещённый графический контент. Это собственный контур контроля банка, за который банк отвечает перед регулятором.

Почему это MLSecOps, а не просто классификатор

Задача – не «обучить модель отличать плохое от хорошего», а построить средство защиты, которое работает против активного нарушителя и встраивается в эксплуатацию. Недопустимый контент появляется двумя путями: случайно (добросовестный пользователь + непредсказуемость модели) и намеренно (пользователь подбирает обходные запросы, грузит состязательно искажённые изображения, эксплуатирует слепые зоны). Решение должно закрывать оба случая.

Полезно в начале зафиксировать модель угроз – от каких атак вы защищаетесь (можно опираться на MITRE ATLAS, OWASP Top 10 for LLM/GenAI).

Что нужно сделать

1. Определить категории запрещенного контента с обоснованием.
2. Провести обзор имеющихся подходов – какие есть готовые открытые модели детекции запрещенного контента, их сильные и слабые стороны.
3. Построить бейзлайн – работающий прототип цензор-модуля.
4. Проверить эффективность по измеримой методике.

К защите команда приносит: работающий прототип (подаешь запрос и/или изображение – получаешь вердикт, категорию и обоснование); документ с таксономией; отчёт по метрикам; краткую модель угроз и список нерешенных рисков; презентацию с демо.

Полезные вопросы для рефлексии в командах:

- Где должен стоять контроль – только на выходе или и раньше? Как сделать цензор-модуль независимым от генератора, чтобы его нельзя было обойти?
- Как ловить не только очевидное, но и случаи, где по отдельности всё «чисто», а вместе – нарушение?

- Чем отличается «поймать всё» от «не блокировать лишнего»? Какая ошибка для банка дороже и как это измерять?
- Что будет, если пользователь специально подбирает обходные запросы? Как проверить устойчивость к таким обходам?
- Можно ли доверять одному детектору или одному провайдерскому фильтру как единственному слою?
- Как объяснить регулятору, почему конкретную картинку заблокировали?

Дополнительные материалы

Исследования/материалы по теме:

1. <https://huggingface.co/google/shieldgemma-2-4b-it>
2. <https://arxiv.org/abs/2406.05113>
3. <https://unsafebench.github.io/>
4. <https://arxiv.org/abs/2510.05173>
5. https://haystack.deepset.ai/cookbook/safety_moderation_open_lms